

Analys av deterministisk design: Vägen till förutsägbara system

1. Introduktion till deterministisk design i ACME

Inom ACME-arkitekturen är **deterministisk design** inte enbart en teknisk målsättning, utan en grundläggande arkitektonisk princip. Det innebär att systemet lämnar kategorin "svart magi" och istället vilar på **matematiska garantier**: givet samma indata och tillstånd ska systemet producera exakt samma utdata, oavsett exekveringsmiljö eller tidpunkt. Genom att eliminera osäkerhetsfaktorer som nätverkslatens och klockvariationer skapar vi ett system där förutsägbarhet är inbyggt i själva fundamentet. Detta angreppssätt adresserar tre strategiska mål:

- **Testbarhet:** Möjligheten att verifiera komplex AI-logik isolerat, utan beroenden till externa API:er eller instabila nätverk.
- **Återspelning (Replay):** Förmågan att med bit-precision återskapa historiska händelseförlopp för att förstå exakt varför ett visst beslut fattades.
- **Offline-kapacitet:** Tillåter utveckling och verifiering av scenarier lokalt med garantin att de beter sig identiskt i produktion. Genom att ersätta slumpmässighet med determinism eliminerar vi "flaky tests" redan vid ritbordet. Denna förutsägbarhet kräver dock att vi kan garantera orubbliga och stabila identiteter genom hela systemet.

2. Identitetens anatomi: Versionerade algoritmer

För att uppnå determinism kan vi inte förlita oss på traditionella metoder som slumpmässiga UUID:n eller tidsstämplar. ACME använder istället versionerade, innehållsadresserade algoritmer (ADR-0004, ADR-0009) för att härleda identiteter. | Algoritmnamn | Syfte | Vad den skyddar mot || ----- | ----- | ----- || acme-cjson-1 | Skapar en kanonisk JSON-sträng av objekt. | Skillnader i fältordning eller formatering som annars skulle korrumpera en hash. || acme-transition-id-1 | Härleder ID för en tillståndsövergång. | Instabila retries; säkerställer att samma logiska operation får samma ID varje gång. || acme-execution-id-1 | Skapar ID för en körning baserat på dess **idempotens-scope**. | Slumpmässighet vid initiering; möjliggör identisk återanvändning av exekverings-ID utan nya resurser. || acme-model-request-hash-1 | Genererar en hash av det exakta anropet till AI-modellen. | Motstridiga tillstånd; upptäcker omedelbart om en prompt har ändrats mellan körningar. |

Viktig insikt: En kritisk aspekt i ACME är att tidsstämplar är **explicit bannlysta** från hash-underlaget för identiteter (ADR-0004). Att inkludera tid eller slumpmässighet i en identitet gör den instabil över tid. Genom att algoritmerna är oföränderliga (**immutable**) skapar vi ankare i systemet som tillåter oss att "resa i tiden" genom replay.

3. Deterministisk Replay: Att återskapa systemets tillstånd

Mekanismen för **Replay** (ADR-0012, ADR-0019) gör det möjligt att verifiera systemets tillstånd genom att använda lagrad evidens istället för att genomföra nya I/O-anrop. Detta förvandlar historisk data till en exekverbar specifikation. Processen för Replay-verifiering sker i fyra steg:

1. **Laddning av evidens:** Systemet läser in den sparade uppgiften, kontexten och de inspelade AI-svaren.

2. **Klock-ersättning:** Den vanliga systemklockan ersätts med en **recorded execution clock** (den lagrade committedAt-tiden). Detta lurar systemet att tro att det är "nu", vilket gör tidsberoende logik fullt testbar.
3. **Isolerad exekvering:** Logiken körs på nytt utan att tillåta ny I/O eller slumpmässiga generatorer.
4. **Verifiering:** Den nya hash-summan för operationen (acme-operation-digest-1) jämförs med den lagrade originalsumman. Status vid en replay-verifiering definieras enligt följande: **Match:** Den rekonstruerade operationen är identisk med den inspelade. **Different:** En **diagnostisk signal** på att koden eller underlaget har ändrats på ett sätt som påverkar resultatet. Det indikerar en regression eller en avsiktlig logikförändring. **Unavailable:** Nödvändig evidens (t.ex. den lagrade AI-responsen) saknas, vilket gör verifiering omöjlig. Denna förmåga att återskapa exakta tillstånd lägger grunden för en testmiljö som är helt bortkopplad från omvärlden.

4. Offline-testning och detektiva mockar

Genom att kombinera deterministiska mock-adaptrar (adapter-model-mock) och ScenarioRunner (ADR-0012) möjliggör ACME testning som producerar **bit-stable evidence** (bit-stabil evidens). **Jämförelse av mock-strategier:**

- **Traditionell mock:** Returnerar ofta svar baserat på ordning (FIFO). Detta leder till **bräckliga tester (brittle tests)** som går sönder vid minsta omstrukturering av koden, även om logiken är korrekt.
- **ACME:s deterministiska mock:** Använder acme-model-request-hash-1. Mocken returnerar endast ett svar om den inkommande prompten matchar den sparade hashen exakt. Eftersom varje ändring i en prompt – ända ner till ett kommatecken – ändrar request-hashen, fungerar mocken som en detektiv. Den flaggar omedelbart om ett kontrakt har brutits. Detta transformerar enkla tester till **självdokumenterande regressionstester** som garanterar systemets resiliens även vid krascher i produktion.

5. Resiliens genom determinism: Durable Resume

Konceptet **Durable execution resume** (ADR-0017) adresserar det kritiska "krasch-fönstret" – perioden mellan att en AI-respons tas emot och att den kommitas till databasen. Tack vare deterministiska ID:n kan ACME återuppta en avbruten körning utan att "betala två gånger" för samma svar. Hanteringen sker i tre steg:

1. **Återanvändning av reservation:** Vid omstart identifierar systemet att en reservation (model:0, attempt 1) redan finns. Systemet återanvänder detta ID istället för att generera ett nytt.
2. **Kontext-omläsning:** Innan exekveringen fortsätter läser systemet om den aktuella kontexten (state/memory). Detta säkerställer **concurrency-kontroll**; om en annan process har skrivit till samma entitet under kraschen, upptäcks detta som en konflikt.
3. **Fullbordan:** Systemet använder den lagrade responsen för att slutföra tolkning och tillståndsförändring utan att belasta AI-providern. **IMPORTANT Resume-garanti:** Vid en resume får systemet aldrig anropa en provider på nytt om en reservation redan finns. Detta förhindrar att vi får ett nytt, annorlunda svar som skulle kunna korrumpera systemets logiska integritet.

6. Slutsats: Värdet av den deterministiska arkitekturen

Deterministisk design är fundamentet som gör att vi kan lita på AI-drivna system. För en arkitekt innebär detta tre avgörande fördelar:

1. **Eliminering av instabilitet:** Inga fler "flaky tests". Verifieringen är binär och matematisk.
2. **Exakt auditering:** Vi kan med absolut precision svara på "varför hände X?" genom att inspektera stabila identiteter och den lagrade evidenskedjan.
3. **Kostnadskontroll:** Genom resume och innehållsadresserad cachning minimeras onödiga API-anrop, vilket direkt påverkar sista raden. Determinism förvandlar AI-integration från osäker "voodoo" till strikt **engineering discipline**, där varje tillståndsovergång är förutsägbar, spårbar och verifierbar.